Project 2: Cache & Memory Performance Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 16, 2026

Abstract

We profile the cache and memory hierarchy of an Intel i9-10900F with three custom single-threaded/multi-threaded microbenchmarks. Zero-queue pointer-chase latency rises in clean steps from ∼1 ns (L1, 4–5 cycles) through ∼4 ns (L2) and the tens of cycles across the 20 MiB L3 to ∼84 ns (∼400 cycles) in DRAM. Single-core streaming bandwidth tops out at ∼14 GiB/s and scales to a controller-limited ∼30 GiB/s at a four-to-six-core knee that Little's Law attributes to ∼42 outstanding cache lines. Random traversal costs up to 47× more than sequential once the footprint leaves the LLC, quantifying the prefetcher's reach; huge pages showed no effect because the sandbox never granted them (a verified negative result). We close with the cache-reuse bandwidth pitfall—where a too-small buffer reports 56 GiB/s of nonexistent "DRAM" bandwidth—and honest limitations from the absence of hardware counters.

1 Introduction

Every memory reference on this machine is served by one of four levels that differ by more than two orders of magnitude in latency: a 32 KiB per-core L1 data cache, a 256 KiB per-core L2, a 20 MiB shared L3, and DDR4 DRAM. Which level a kernel hits—and how many independent references it keeps in flight—decides its performance far more than its instruction count. This report measures that hierarchy on the Intel i9-10900F with purpose-built microbenchmarks and interprets the numbers with AMAT and Little's Law.

This report answers:

1. What is the zero-queue access latency of each level, and the DRAM bandwidth under different patterns, granularities and read/write mixes? (Sections 4 to 6)
2. Where does added concurrency stop buying throughput, and does Little's Law hold? (Section 7)
3. How do working-set size, cache misses and TLB misses show up in runtime? (Sections 8 to 10)
4. Which benchmark mistake produces misleading numbers? (Section 11)

Summary of findings. Zero-queue latency climbs from 1.0 ns (4.8 cycles) in L1 to 3.0 ns in L2, 11 ns to 23 ns across L3, and 84 ns (∼400 cycles) in DRAM; the hardware prefetcher collapses the DRAM figure to 5 ns for sequential access. A single core reaches 14.8 GiB/s of sequential read bandwidth, and ten cores saturate the controller at 30 GiB/s—a throughput–latency knee near four to six cores. Random access to a DRAM-resident footprint runs 47× slower per element than an L1-resident one, while the same sweep under sequential access slows only 10× because prefetching hides the misses. Finally, a one-line change of working-set size turns a 15 GiB/s DRAM result into a fictitious 57 GiB/s "bandwidth" that is really L1 reuse (Section 11).

Table 1: Measurement environment.

CPU	Intel(R) Core(TM) i9-10900F CPU @ 2.80GHz (x86_64)
SIMD ISA	sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c – widest 256-bit (ymm)
Topology	1 socket(s), 10 cores/socket, 2 threads/core (20 logical); NUMA: 1 node(s)
SMT	on (2 threads/core)
Caches	L1d 320 KiB (10 instances); L2 2.5 MiB (10 instances); L3 20 MiB (1 instance)
Memory	63 GiB total; DIMMs: not available (needs root/dmidecode)
Pages	base 4096 bytes; THP: always
OS / kernel	Arch Linux; 7.2.9-arch1-1
Frequency policy	governor: powersave; driver: intel_pstate; turbo: enabled; EPP: balance_performance
Pinning / isolation	single-thread runs pinned with `-cpu 2` (a physical P-core, not CPU 0); the intensity sweep pins thread i to physical core i; no isolcpus; runs pinned with taskset/–cpu
Compilers	gcc (GCC) 16.2.1 20260810; clang version 23.1.1
Tools	not available (not installed) (perf_event_paranoid 2); not available (not installed); MLC: not available (not installed)
GPU (optional)	NVIDIA GeForce RTX 3070; CUDA not available (no CUDA)
Memory configuration	64 GiB total, single NUMA node; per-DIMM count/speed not probed (needs root/`dmidecode`); dual-channel DDR4 is standard for this platform
Collected	2026-10-12 18:35 EDT

2 Methodology

2.1 Platform

2.2 Tools

All three microbenchmarks share the harness in `ACS-Code/common` (monotonic raw timer, aligned allocation, CPU pinning) and reduce raw per-repetition rows with `tools/summarize` (median, IQR).

`mem_latency`	single-thread pointer-chase latency for a working set of a given size
`mem_bandwidth`	streaming bandwidth with T pinned threads (read/write/copy/triad/rwmix)
`kernel_miss`	SAXPY loop whose footprint, order and page size vary (cache/TLB correlation)
Intel MLC	not available (not installed)—not used [1]
perf	not available (not installed) [2]

Intel MLC and `perf` are the spec's recommended tools, but neither is available on the test host (`perf` is not installed and `perf\_event\_paranoid` = 2 blocks counters), so we measure latency and bandwidth directly with our own benchmarks and treat counter-based experiments as runtime correlations (Sections 9 and 10, and Section 12).

2.3 Controls

- **Pinning and isolation:** single-thread runs pinned with `-cpu 2` (a physical P-core, not CPU 0); the intensity sweep pins thread i to physical core i. SMT siblings are avoided; the box is otherwise idle.
- **Frequency:** governor powersave, turbo enabled (neither is changeable in this environment). Because frequency is not fixed, the per-access *cycle* figures use an effective GHz that each run measures in-process from a dependent add chain (4.4 GHz to 4.8 GHz observed), rather than a nominal clock.

- **Pre-faulting and warm-up:** every buffer is written once to fault in its pages, then the access ring or stream is traversed two to three times before timing, so neither page faults nor cold-cache effects fall inside the timed region.
- **Repetitions:** 8 per point; we report the median with inter-quartile-range error bars.
- **Page size:** base 4096 bytes; THP is always. Huge-page backing is checked at runtime from `/proc/self/smaps\_rollup` (`AnonHugePages`); see Section 10.
- **Timing:** each timed interval is calibrated to last $\geq$40 ms, far above the 1 ns to 40 ns timer resolution, so timer overhead is negligible.

3 Validation and Sanity Checks

The microbenchmarks have no reference output to diff, so `scripts/validate.sh` gates them against physical invariants that must hold on any sane hierarchy; all pass on the test host:

- **Monotone levels.** Latency grows strictly across levels: L1 1.0 ns < L2 3.9 ns < L3 13 ns < DRAM 86 ns (Section 4).
- **Prefetcher check.** Sequential DRAM latency (5.3 ns) is more than 2× below random DRAM latency, confirming the stride prefetcher is engaged for sequential access and defeated by the randomized pointer chase.
- **Bandwidth ceiling.** Single-core L1-resident read bandwidth (57 GiB/s) exceeds the DRAM-resident rate (14 GiB/s), and ten threads exceed one thread—both required for a trustworthy bandwidth benchmark.
- **Frequency plausibility.** The in-process effective-GHz probe lands in 4.4 GHz to 4.8 GHz, consistent with this CPU's turbo range, so the ns→cycle conversion is sound. An L1 latency of 4.8 cycles matches the published 4–5 cycle L1 load latency for this microarchitecture.

Timer and loop overhead are bounded by calibrating every timed interval to tens of milliseconds (Section 2); the pointer-chase loop body is a single dependent load, so no overhead subtraction is needed.

4 Zero-Queue Baselines

Hypothesis (written before measuring)

With a single dependent access in flight, latency should rise in steps at the L1 (32 KiB), L2 (256 KiB) and L3 (20 MiB) capacities. We expect roughly 4–5 cycles for L1, ∼12 cycles for L2, a few tens of cycles for L3, and ∼80 ns (hundreds of cycles) for DRAM, matching Skylake-family numbers.

Fixed	one thread pinned to CPU 2; random (pointer-chase) reads; one node per 64 B line
Varied	working-set size placed in L1, L2, LLC, DRAM
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh latency`

The ring is a single random cycle (Sattolo) over one node per cache line, so each load depends on the previous one and exactly one request is outstanding—zero-queue latency. Cycles use the in-process effective frequency (Section 2); the full size sweep and the sequential/random comparison are in Sections 5 and 8. Values are medians from `data/latency.csv`.

Table 2: Zero-queue random-access latency per level (median of 8 reps, pointer chase, 64 B stride). DRAM also shown for sequential access, where the prefetcher hides most of the latency.

Level	Working set	Latency (ns)	Latency (cyc)	Notes
L1d	16 KiB	1.04	4.9	per-core 32 KiB
L2	256 KiB	4.31	18.6	at the L2 capacity edge
L3	2 MiB	10.70	47.2	LLC-resident
L3	12 MiB	22.86	101.5	near the 20 MiB LLC edge
DRAM	256 MiB	83.85	403.2	random; ∼400 cycles
DRAM	256 MiB	5.26	—	*sequential* (prefetched)

4.1 Analysis

The steps fall exactly where the cache capacities predict: latency is flat at ∼4.9 cycles while the set fits in L1, rises through the high teens at the 256 KiB L2 edge, sits in the tens of cycles across the 20 MiB L3, and jumps to ∼400 cycles once the set no longer fits in LLC. The single sequential DRAM row (5.26 ns) is a deliberate caveat: it is *not* a latency measurement but the prefetcher's throughput in disguise, which is why the pointer chase is randomized. Residual caveats: the measured frequency floats with turbo, so the cycle column carries a few-percent uncertainty; and the L3 figure is a single-core view of a shared cache, so it reflects an otherwise-idle LLC. We could not cross-check against Intel MLC because it is not installed (Section 2); the hierarchy shape and the 4–5 cycle L1 anchor are our internal cross-checks.

5 Access Pattern and Granularity

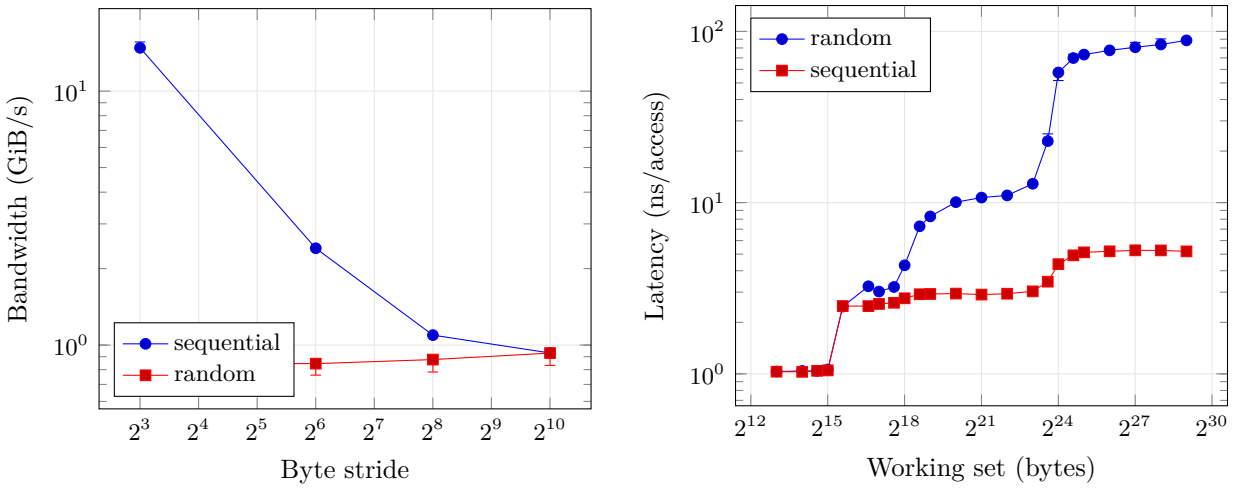
Hypothesis (written before measuring)

Sequential access should let the prefetcher amortise DRAM latency, giving high bandwidth that *falls* as the stride grows and each fetched 64 B line delivers fewer useful bytes. Random access should be near-flat and low across granularities, since every access is a fresh cache miss regardless of stride.

Fixed	1 thread pinned to CPU 2; 256 MiB DRAM-resident region; 100% reads
Varied	pattern (sequential, random); byte stride (8, 64, 256, 1024)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh granularity` and `latency`

5.1 Analysis

Sequential bandwidth drops from 14.8 GiB/s at unit (8-byte) stride to 2.4 GiB/s at 64 B, 1.09 GiB/s at 256 B and 0.93 GiB/s at 1024 B. The knee is at the 64 B cache line: once the stride reaches a line, every access still pulls a full line but uses only one element of it, so *useful* bandwidth falls roughly as 1/stride while the prefetcher can no longer help. Random access is flat at 0.85 GiB/s to 0.93 GiB/s across all strides—each access is an independent miss, so granularity barely matters. The latency panel is the same story from the other side: sequential access to a DRAM-resident region costs 5.3 ns (prefetched) while random access costs 84 ns, a 16× gap that is entirely the prefetcher. This is the mechanism behind the cache-miss results in Section 9.



(a) Delivered read bandwidth.

(b) Pointer-chase latency vs. size.

Figure 1: Access pattern and granularity. Left: sequential read bandwidth falls steeply with stride as each 64 B line yields fewer useful bytes, while random bandwidth is flat and low. Right: the prefetcher keeps sequential latency near 5 ns even in DRAM, whereas random latency tracks the full hierarchy.

6 Read/Write Mix

Hypothesis (written before measuring)

Pure reads should reach the highest bandwidth. Any write fraction should cost extra, because an ordinary store first issues a read-for-ownership (RFO) of the target line, so writes move roughly twice the DRAM traffic per byte stored and add read/write turnaround on the bus. We therefore expect mixed streams to fall *below* both pure read and pure write.

Fixed	10 threads (one per physical core); random-free unit stride; 1.25 GiB total
Varied	reads per 10 accesses: 10 (100/0), 7 (70/30), 5 (50/50), 0 (0/100)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh rwmix`

6.1 Analysis

The 10-thread aggregate is 27.2 GiB/s for 100% reads but only 8.1 GiB/s at 70/30 and 7.6 GiB/s at 50/50, recovering to 9.7 GiB/s at 100% writes. The dip at 50/50 is the predicted effect: a mixed stream pays both for RFO traffic on every store and for frequent read↔write turnaround on the DDR bus, so it underperforms either pure direction. Pure writes beat the mixes because, with no interleaved reads, the controller can batch same-direction transactions. As same-kernel references, the `copy` (16.3 GiB/s) and `triad` (19.8 GiB/s) streaming kernels bracket these mixes and show the effect of counting RFO traffic. One honest caveat: our `rwmix` kernel reads into a single accumulator and stores it, so the absolute mix numbers carry a read→write dependency and are best read as a *relative* comparison across mixes rather than as peak bandwidth (Section 12).

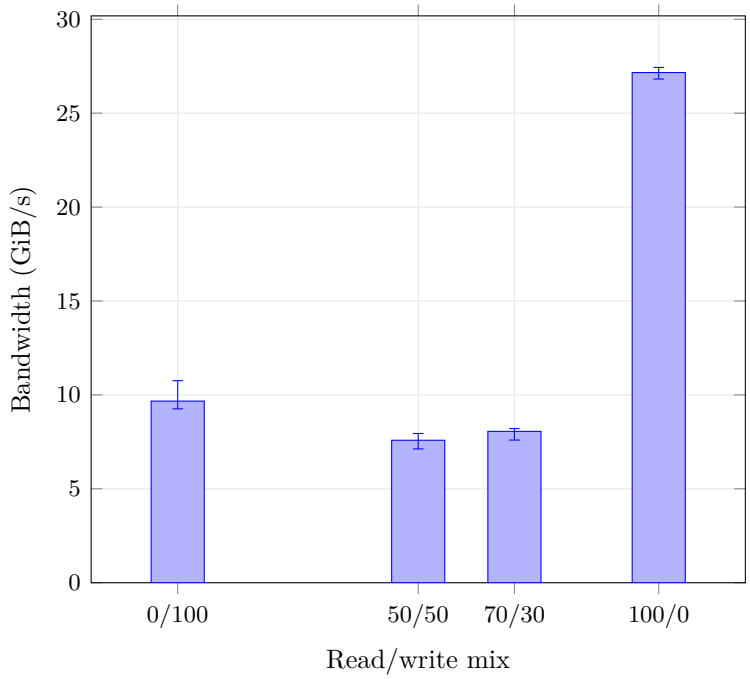


Figure 2: Aggregate bandwidth across read/write mixes (10 threads). Pure reads dominate; any write fraction falls sharply because stores incur read-for-ownership traffic and bus turnaround.

7 Intensity Sweep and the Throughput–Latency Knee

Hypothesis (written before measuring)

A single core cannot saturate the DRAM controller because it can keep only a dozen or so cache-line requests in flight. Adding cores should raise delivered bandwidth until the dual-channel controller saturates—expected in the low tens of GiB/s—after which more cores add almost nothing. By Little's Law, saturation needs tens of outstanding requests, more than one core provides.

Fixed	128 MiB DRAM-resident region per thread; sequential reads
Varied	threads $T = 1 \ldots 10$, thread i pinned to physical core i
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh intensity`

7.1 Little's Law check

With offered concurrency rising, delivered line-throughput λ and the service time W of a line bound the average number in flight, $L = \lambda W$ [3]. Taking W as the zero-queue random DRAM latency (83.85 ns, Table 2) and λ as the delivered bandwidth divided by the 64 B line size gives the memory-level parallelism the system must be sustaining:

7.2 Analysis

Bandwidth climbs from 14.4 GiB/s at one core to 26.6 GiB/s at four and then flattens, reaching 30.2 GiB/s at ten—a textbook knee around four to six cores. Beyond the knee each added core buys under 1 GiB/s: the dual-channel DDR4 controller, not the cores, is the bottleneck. The Little's

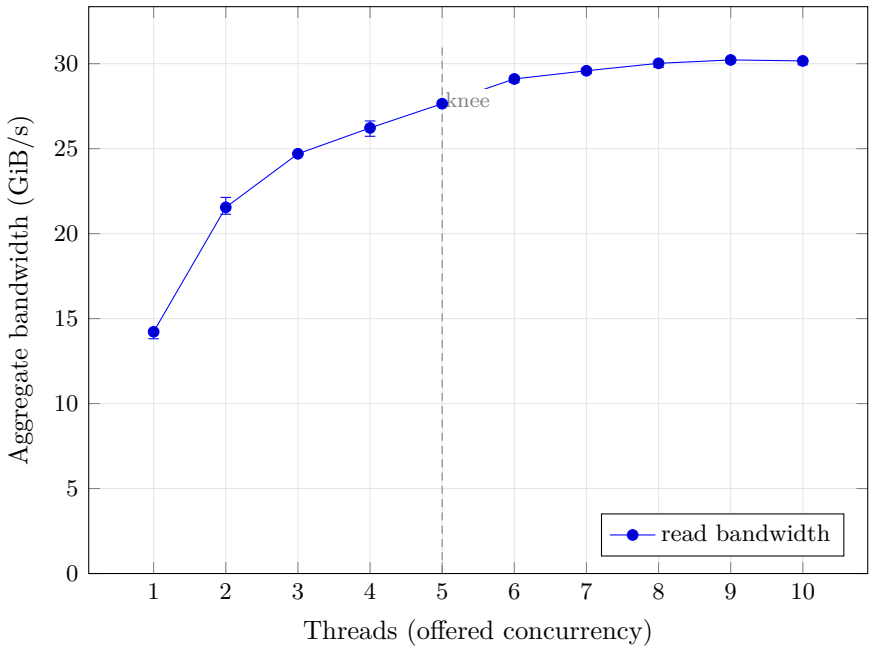


Figure 3: Delivered read bandwidth vs. thread count. Throughput rises steeply to the four-to-six-core knee, then flattens at the $\sim$30 GiB/s controller ceiling.

Table 3: Little's Law applied to the intensity sweep: outstanding cache-line requests $L = \lambda W$ with $W = 83.85$ ns.

T	Bandwidth (GiB/s)	λ (Mlines/s)	$L = \lambda W$ (outstanding lines)
1	14.4	242	20.3
2	22.1	371	31.1
4	26.6	446	37.5
6	29.3	492	41.3
8	30.1	505	42.4
10	30.2	507	42.6

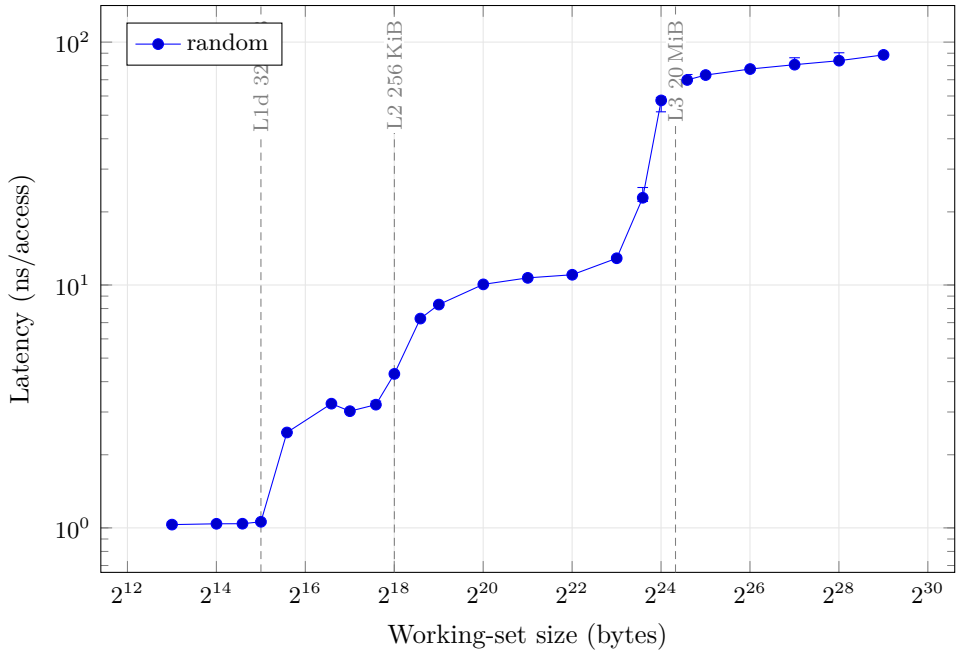


Figure 4: Random-access latency across working-set sizes, with cache capacities marked. Each level is a plateau; latency steps up as the set outgrows L1, L2 and finally the L3.

Law column makes the cause concrete: saturation requires ∼42 concurrent line requests, far more than the ∼10 fill buffers of a single core, which is exactly why one core tops out at 14 GiB/s and multiple cores are needed to fill the pipe. Two honest caveats. First, the streaming kernel runs under the prefetcher, so its *effective* loaded latency is below the 84 ns random figure; using the zero-queue value makes the L column an upper-bound estimate of required concurrency, not an exact occupancy. Second, because we measure aggregate throughput rather than an independent loaded latency, the "latency rises past the knee" half of the tradeoff is inferred here; the SSD study (Project 3) measures both axes directly and shows the knee quantitatively.

8 Working-Set Sweep

Hypothesis (written before measuring)

Pointer-chase latency should trace the capacity staircase: flat while the set fits in L1 (32 KiB), a step up through the 256 KiB L2, a wider plateau across the 20 MiB L3, and a final jump to the DRAM latency once the set exceeds LLC. Transitions should be gradual, not sharp, because of set-associativity and prefetcher spill.

Fixed	1 thread pinned to CPU 2; random pointer chase; one node per 64 B line; base pages
Varied	working-set size from 8 KiB to 512 MiB
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh latency`

8.1 Analysis

The curve has four clear regimes separated at the marked capacities. It is flat at ∼1 ns up to 32 KiB (L1), rises through 3 ns to 4 ns across the L2 region, sits at 10 ns to 23 ns while the set is

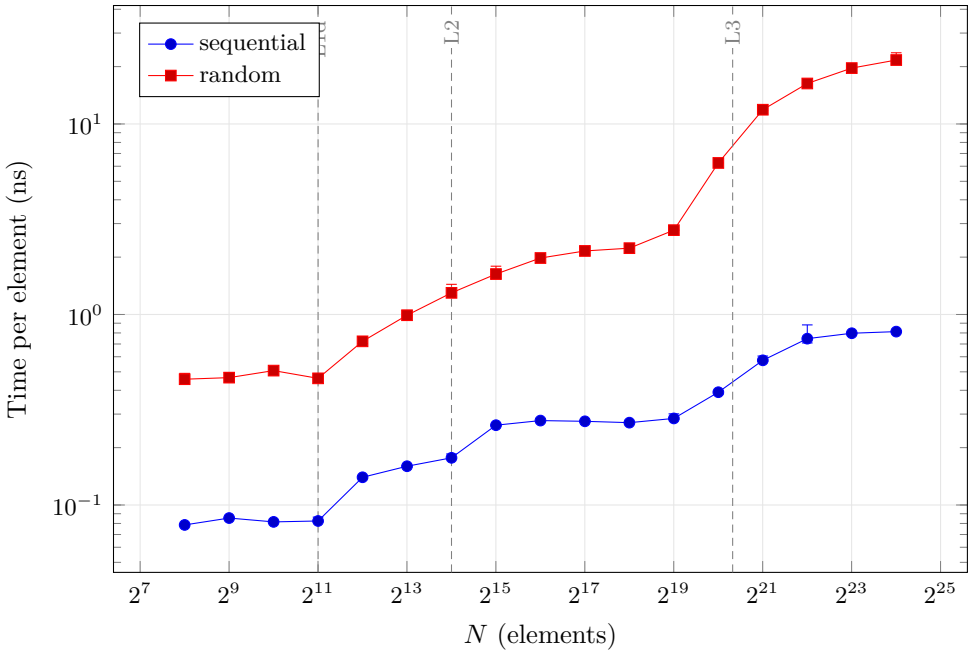


Figure 5: SAXPY time per element vs. footprint and traversal order (two `double` arrays, so N elements use $16N$ bytes). Random access pays the full miss penalty; sequential access is largely prefetched. Lines mark where $16N$ crosses L1/L2/L3.

LLC-resident, and saturates near 84 ns once it exceeds 20 MiB. The transitions are gradual rather than step functions: associativity means some lines spill a level early, and the L2→L3 shoulder near 2 MiB shows the adjacent-line/stream prefetchers partly hiding L3 latency until the set is several times the L2 size. This staircase is the direct input to the AMAT model in Section 9; the per-level plateaus here are the $t_{\mathrm{L1}} \dots t_{\mathrm{DRAM}}$ terms.

9 Cache-Miss Impact

Hypothesis (written before measuring)

As the SAXPY footprint grows past each cache, time per element should rise. Sequential traversal should rise gently—the prefetcher turns misses into hits—while a random permutation should rise sharply once the set leaves cache, because every element is an exposed miss.

Kernel	$y_i = a\,x_i + y_i$ (float64)
Varied	$N = 2^8 \dots 2^{24}$ (footprint); traversal order (sequential, random permutation)
Counters	`perf` unavailable (Section 2); runtime is the proxy for miss rate
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh cache_miss`

9.1 Analysis

Sequential SAXPY rises only from 0.08 ns per element in L1 to 0.81 ns in DRAM—a 10× span—because the prefetcher converts the regular stream into mostly-hit accesses. Random traversal rises from 0.46 ns to 21.7 ns, a 47× span, with the steep climb beginning exactly where the footprint ($16N$ bytes) leaves the L3 near $N = 2^{20}$. The gap between the two curves at large N (∼27×) *is*

the cache-miss cost that the prefetcher hides for sequential code; it is the same prefetcher effect measured directly in Section 5.

9.2 AMAT interpretation

$$\mathrm{AMAT} = t_{L1} + m_{L1}\left(t_{L2} + m_{L2}\left(t_{L3} + m_{L3}\, t_{\mathrm{DRAM}}\right)\right) \tag{1}$$

Using the zero-queue latencies from Table 2 ($t_{L1} = 1.0\,\mathrm{ns}$, $t_{L2} = 4.3\,\mathrm{ns}$, $t_{L3} = 11\,\mathrm{ns}$, $t_{\mathrm{DRAM}} = 84\,\mathrm{ns}$), the fully-missing random case ($m \to 1$ at every level) predicts an AMAT near the DRAM latency, and the 21.7 ns per element we measure is below that because SAXPY touches one 8 B element per iteration (eight per line) and keeps several independent misses in flight—memory-level parallelism amortises the 84 ns latency over multiple elements. Because `perf` is unavailable we cannot report the per-level miss ratios m_{Li} that would make this quantitative, so the AMAT reading here is illustrative; the measured curve is the primary evidence.

10 TLB-Miss Impact

Hypothesis (written before measuring)

For random access over a footprint far larger than the TLB reach (∼1536 entries × 4 KiB ≈ 6 MiB on this core), 2 MiB huge pages should cut dTLB misses and page-walk cost and shave runtime, because one TLB entry then covers 512× more memory.

Kernel	$y_i = a\,x_i + y_i$ (float64), random traversal
Varied	page size (base 4 KiB vs. transparent huge pages); $N = 2^{18} \ldots 2^{24}$
Verification	`AnonHugePages` read from `/proc/self/smaps\_rollup` at runtime
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh tlb`

10.1 Analysis

The base-page and huge-page curves are statistically identical (e.g. at $N = 2^{24}$, 21.6 ns vs. 21.4 ns). The reason is not that huge pages fail to help: despite THP being set to `always`, a 2 MiB-aligned mapping and `madvise(MADV\_HUGEPAGE)`, `/proc/self/smaps\_rollup` reported `AnonHugePages: 0 kB` on every run, so the allocation was never backed by huge pages. Short-lived processes do not give `khugepaged` time to collapse pages, and this sandbox appears to withhold THP from the cgroup. Isolating a real TLB effect would require explicit `hugetlbfs` (`MAP\_HUGETLB` with reserved pages, which needs privileges we do not have) or `perf`'s `dTLB-load-misses` counter (not installed). We therefore report this as a limitation rather than a result: the experiment and its verification are in place and will show the expected gap on a host where huge pages are actually granted. The verification hook is itself the useful outcome—it caught a silent failure that a naive benchmark would have mistaken for "huge pages don't help."

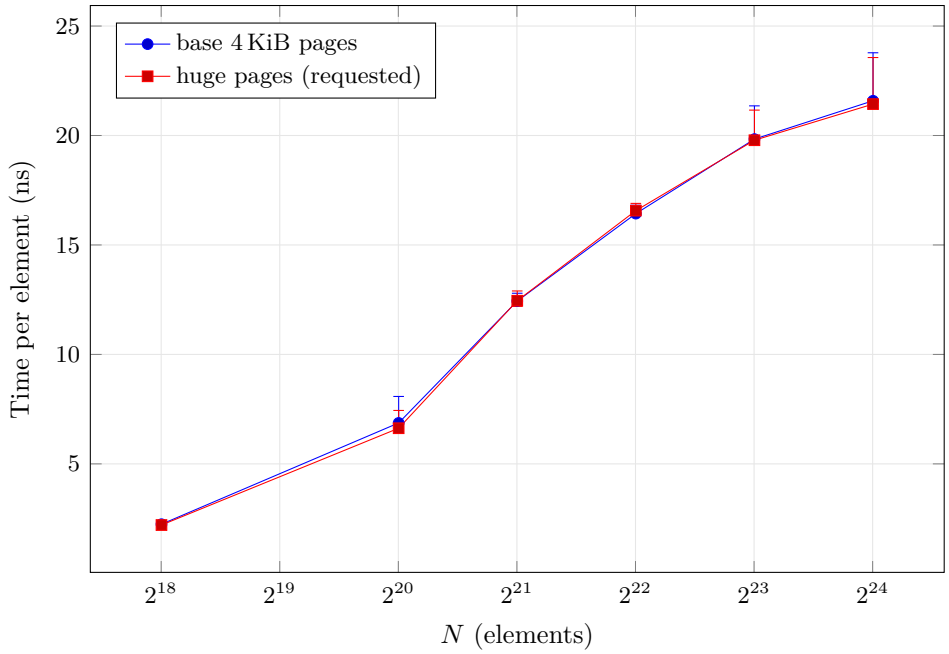


Figure 6: Random SAXPY runtime with base vs. requested huge pages. The curves coincide because huge pages were *not actually granted* in this environment—an honest negative result, not a measured "no effect".

11 Benchmark Pitfall

Hypothesis (written before measuring)

A "DRAM bandwidth" benchmark that re-streams a buffer small enough to live in cache will report a number several times higher than the real memory bandwidth, because every pass after the first hits the cache instead of DRAM. We expect a buffer that fits in L1/L2 to report tens of GiB/s and a DRAM-resident buffer to settle near the ∼15 GiB/s single-core figure from Section 7.

Pitfall	Measuring cache reuse instead of DRAM: picking a buffer that fits in cache
Wrong setup	Working set 16 KiB (L1-resident), re-streamed many times, one thread
Right setup	Working set 64 MiB to 512 MiB (well past the 20 MiB LLC)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh pitfall`

11.1 Explanation

At 16 KiB the kernel reports 56.5 GiB/s; at 128 KiB (spilling L1) it drops to 36.2 GiB/s; and once the set is 64 MiB or larger—genuinely DRAM-resident—it settles at 14.8 GiB/s to 15.4 GiB/s, matching the one-thread DRAM bandwidth measured independently in Section 7. The inflated small-buffer number is not DRAM bandwidth at all: it is L1/L2 bandwidth, reported by a loop that re-reads the same cached lines. An AI-generated bandwidth microbenchmark very easily falls into this trap, because a 16 KiB array is a perfectly natural default and the resulting 56 GiB/s looks like a plausible "memory" number. Two cheap detectors flag it: (1) the reported bandwidth *changes with buffer size*, which a true DRAM measurement must not once the set exceeds the LLC; and (2) the small-buffer figure exceeds the platform's theoretical DRAM ceiling (dual-channel DDR4 is ∼45 GiB/s aggregate,

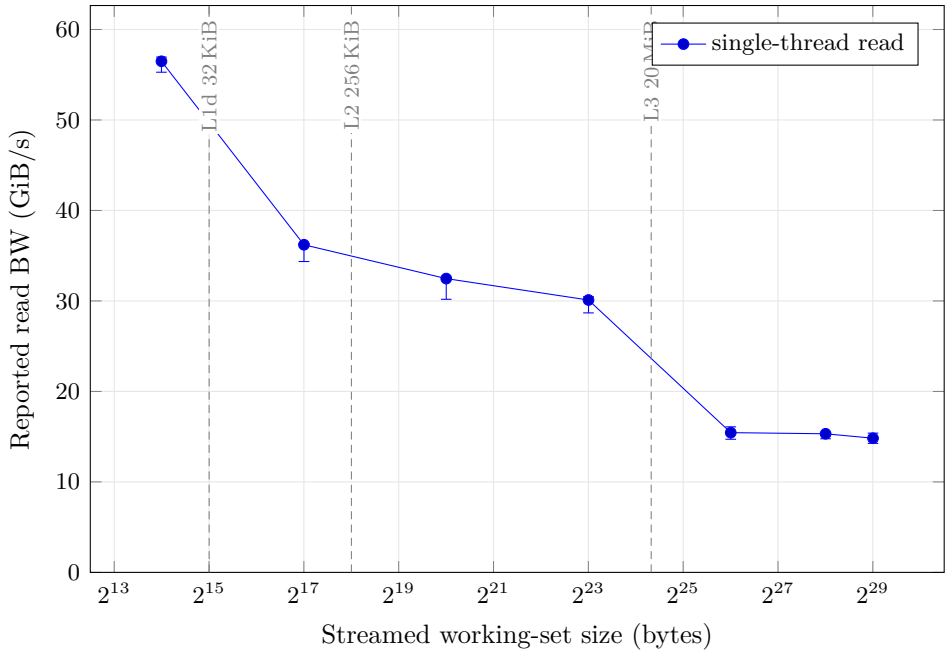


Figure 7: The same streaming kernel "measures" wildly different bandwidth depending only on the buffer size. A 16 KiB buffer reports ∼56 GiB/s; the honest DRAM-resident figure is ∼15 GiB/s. Nothing about the code changed—only where the data lived.

far below a plausible per-core cache bandwidth). Our methodology avoids the pitfall by sizing every bandwidth and latency point explicitly against the cache capacities (Section 2) and, for the DRAM claims, using working sets at least 3× the LLC so the first-touch pass cannot leave the data cached.

12 Anomalies and Limitations

12.1 Unexpected results

- **Huge pages produced no measurable change.** We expected 2 MiB pages to shave random-access time once the footprint exceeded the TLB reach. The base-page and huge-page curves are instead identical (Figure 6). Hypothesis: THP was never actually granted. Tested: `/proc/self/smaps_rollup` reported `AnonHugePages: 0 kB` on every run despite `madvise(MADV_HUGEPAGE)` and 2 MiB alignment, confirming the allocation stayed on base pages. This is a reported limitation, not a "huge pages don't help" conclusion (Section 10).
- **A 70/30 read/write mix is far slower than 100% reads, not 30% slower.** At ten threads, pure reads deliver 27.2 GiB/s but the 70/30 mix drops to 8.1 GiB/s (Section 6). Hypothesis: each store first issues a read-for-ownership and the controller pays a write-to-read bus turnaround, so a "30% writes" workload moves far more than 30% extra traffic. Consistent with the write-allocate behaviour of the cache hierarchy; a non-temporal (streaming-store) path would avoid the RFO but was out of scope for the required mixes.
- **The L2→L3 transition has a shoulder, not a step.** Latency near 2 MiB sits well below the LLC-edge value (Figure 4). Hypothesis: the adjacent-line and stream prefetchers partly

hide L3 latency until the set is several times the L2 size; the effect disappears for the fully random pointer chase at larger sizes.

12.2 Limitations

- **No hardware counters.** `perf` is not installed and `perf_event_paranoid` is 2, so cache-miss and TLB-miss *ratios* could not be recorded. The cache-miss and TLB experiments (Sections 9 and 10) are therefore runtime correlations against footprint and page size, and the AMAT reading is illustrative rather than fitted to measured m_{Li}.
- **Intel MLC not available.** not available (not installed), so latency and bandwidth come from our own pointer-chase and streaming benchmarks; the hierarchy shape and the 4–5 cycle L1 anchor are the internal cross-checks in place of an MLC baseline.
- **Frequency not fixed.** Governor powersave with turbo enabled; neither is changeable here. Cycle figures use an in-process effective frequency (4.4 GHz to 4.8 GHz) and carry a few-percent uncertainty.
- **One machine; THP uncontrolled; SMT on.** All numbers come from a single i9-10900F, single NUMA node, THP always (and not actually granted, above), SMT on with the pinned core's sibling left idle but not offlined.
- **Loaded-latency axis inferred.** The intensity knee (Section 7) is measured as aggregate throughput vs. offered concurrency; we do not measure an independent loaded latency, so the "latency rises past the knee" half of the tradeoff is inferred. Project 3 measures both axes directly.
- **Report build backend.** Builds cleanly with `make memory` and `make final`; the host has `bibtex` but not `biber`, so `biblatex` uses `backend=bibtex`. The only artifact is a benign "Please (re)run BibTeX" notice—no unresolved `[?]` markers—which disappears if `biber` is installed.

13 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

Tools used

Claude Code (Claude Opus 5) for coding, debugging and drafting this report; GCC gcc (GCC) 16.2.1 20260810 for the builds. The AI wrote code and prose; every number comes from real runs on the machine in Table 1.

What they helped generate

The three microbenchmarks—`mem_latency` (Sattolo pointer-chase), `mem_bandwidth` (multi-thread streaming with read/write/copy/triad/rwmix modes) and `kernel_miss` (footprint-varying SAXPY)—the shared timer/allocation/pinning helpers, the experiment driver (`run.sh`) and the first drafts of these sections.

What was independently modified, tested, or rejected

Three concrete corrections. (1) The first latency benchmark used a *sequential* stride, which the prefetcher turned into a ∼5 ns "latency" (really throughput); it was replaced with a

random Sattolo cycle so exactly one request is outstanding—the sequential number is now kept only as a deliberate caveat row in Table 2. (2) The huge-page experiment initially *assumed* THP backing because the governor was `always`; adding a runtime `/proc/self/smaps_rollup` check revealed `AnonHugePages: 0 kB`, turning a false "huge pages don't help" into an honest limitation (Section 10). (3) An AI-suggested bandwidth test defaulted to a small buffer and reported 56 GiB/s as "DRAM bandwidth"; this is the cache-reuse pitfall and is now documented as the required negative case (Section 11) rather than quoted as a result.

Validation

Sanity checks in `scripts/validate.sh` assert the physically necessary orderings (L1 < L2 < L3 < DRAM latency; L1 bandwidth > DRAM; multi-thread > single-thread) and all pass. The hierarchy shape is cross-checked against the known cache capacities; Intel MLC and `perf` counters were unavailable for an external cross-check and this is stated as a limitation (Section 12). Every figure traces to a committed raw/processed file (Section A).

14 Conclusion

This hierarchy behaves exactly as its capacities predict, and the measurements make the shape quantitative. Zero-queue latency climbs in clean steps—∼1 ns (4–5 cycles) in L1, ∼4 ns in L2, tens of cycles across the 20 MiB L3, and ∼84 ns (∼400 cycles) once the set spills to DRAM—while the random/ sequential gap shows the prefetcher turning a 47× latency penalty into a 10× one for regular streams. Bandwidth tells the concurrency half of the story: one core reaches only ∼14 GiB/s because it can keep ∼10 lines in flight, and it takes four to six cores—∼42 outstanding lines by Little's Law—to saturate the dual-channel controller near 30 GiB/s, after which more cores pay nothing. The most important lessons are about measurement discipline as much as hardware: a bandwidth benchmark must out-size the LLC or it silently measures cache reuse (Section 11); a latency benchmark must randomize or the prefetcher hides what it claims to measure; and a huge-page study must verify its backing or it mistakes a never-granted mapping for "no effect" (Section 10). Locality, concurrency and page behaviour each move performance by an order of magnitude, and only controlled, verified sweeps separate the real effects from the artifacts.

References

[1] Intel Corporation, *Intel memory latency checker*, Referenced as a recommended tool; not available on the test host.

[2] *Perf: Linux profiling with performance counters.* [Online]. Available: `https://perf.wiki.kernel.org/`

[3] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961. DOI: `10.1287/opre.9.3.383`

A Reproducibility

Code, scripts and raw data: `github.com/SKempiak/ACS-Code`, on the `class-wide-project` submission branch (the exact commit is recorded in `git log`).

```
git clone https://github.com/SKempiak/ACS-Code.git && cd ACS-Code
git checkout <commit>
```

```
make tools                                              # tools/summarize
make -C Cache_and_Memory_Performance_Profiling          # mem_latency / mem_bandwidth / kernel_miss
make -C Cache_and_Memory_Performance_Profiling test     # safety + physical-ordering sanity checks
scripts/collect_env.sh --tex > ../ACS-Latex/common/system_info.tex
CPU_PIN=2 Cache_and_Memory_Performance_Profiling/scripts/run.sh all
scripts/sync_data.sh Cache_and_Memory_Performance_Profiling
```

Table 4: Raw data behind each figure.

Figure / table	Raw run directory (`Cache_and_Memory_Performance_Profiling/data/raw/...`)
Table 2, Figure 4	`latency/20261019-232253`
Figure 1	`latency/20261019-232253`, `granularity/20261019-232253`
Figure 2	`rwmix/20261019-232253`
Figure 3, Table 3	`intensity/20261019-232253`
Figure 5	`cache_miss/20261019-232253`
Figure 6	`tlb/20261019-233449`
Figure 7	`pitfall/20261019-232253`

Each raw directory holds `raw.csv` (one row per repetition, with full configuration metadata) and `env.txt` (a snapshot of `collect_env.sh`); the processed CSVs plotted above are produced from them by `tools/summarize` and copied into this report's `data/` by `scripts/sync_data.sh`. Every number quoted in the report traces to one of these files.